



Lecția 13 – Python

Trainer: Hrișcă Miruna

Aplicații folosite + alternative ▾

01

PyCharm

[Link - windows](#)

[Link - Apple](#)

02

One Compiler

[Link](#)

03

VS Code

[Link](#)

04

Online GDB

[Link](#)

Biblioteca Tkinter



- ❖ Este un **modul special** din Python.
- ❖ Ne ajută să facem **ferestre frumoase**, nu doar programe în consolă (cele negre cu scris alb).
- ❖ Cu Tkinter putem pune:
 - **butoane**  (pe care să dăm click)
 - **spații pentru text**  (unde putem scrie)
 - **canvas**  (o „foaie de desen” unde putem face forme și desene)



Proiect Minge

- Vom realiza o minge colorată care se va mișca aleatoriu pe canva. Dacă va atinge marginea, va devia.



```
import tkinter as tkr
import time
```

- `tkinter` este biblioteca care ne ajută să facem ferestre cu desen (grafice)
- `as tkr` înseamnă că îi spunem „prescurtare”: în loc să scriem `tkinter` de fiecare dată, scriem `tkr`. E doar ca atunci când îi spui prietenului „Alexandru” pe scurt „Alex”. E convenabil și scurtează codul.
- `import time` este pentru lucruri legate de timp



```
import tkinter as tkr
import time
```

- `tkinter` este biblioteca care ne ajută să facem ferestre cu desen (grafice)
- `as tkr` înseamnă că îi spunem „prescurtare”: în loc să scriem `tkinter` de fiecare dată, scriem `tkr`. E doar ca atunci când îi spui prietenului „Alexandru” pe scurt „Alex”. E convenabil și scurtează codul.
- `import time` este pentru lucruri legate de timp

Proiect Minge

```
tk = tkr.Tk()
canvas = tkr.Canvas(tk, width=300, height=400)
canvas.grid()
```

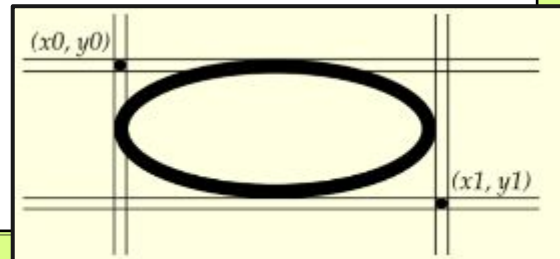


- `tkr.Tk()` creează o fereastră (gândește-te la ea ca la o foaie goală). Am pus variabila `tk` (prescurtare pentru `tkinter.Tk()`), e „fereastra principală”.
- `Canvas` este o zonă în care putem desena forme (cercuri, pătrate etc.). Aici e lățimea 300 pixeli și înălțimea 400 pixeli
- `canvas.grid()` pune acea foaie/tabla în fereastră (o afișează).

```
ball = canvas.create_oval(5,5,60,60, fill="light blue")
```



- `create_oval(5,5,60,60)` desenează un oval/cerc în interiorul unei „cutii” cu colțul din stânga-sus la `(5,5)` și colțul din dreapta-jos la `(60,60)`. Aceasta „cutie” definește locul și mărimea cercului.
- `fill="light blue"` e culoarea bilei.



Proiect Minge

$x = 1$

$y = 3$

x și y sunt „pași” cu care mutăm bila la fiecare cadru (frame).

- $x = 1$ înseamnă că la fiecare pas bila se mișcă 1 pixel spre dreapta (sau -1 spre stânga).
- $y = 3$ înseamnă că la fiecare pas bila se mișcă 3 pixeli în jos (sau -3 în sus).

Gândește-te la ele ca la viteza pe orizontală și verticală.



Proiect Minge



```
while True:
    canvas.move(ball, x, y)
    pos = canvas.coords(ball)
    if pos[3] >= 400 or pos[1] <= 0:
        y = -y
    if pos[2] >= 300 or pos[0] <= 0:
        x = -x
    tk.update()
    time.sleep(0.025)
    pass
```

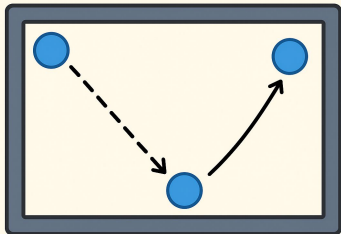
- `while True:` — repetăm la nesfârșit (bucă infinită). E ca o roată care nu se oprește decât dacă oprim programul.
- `canvas.move(ball, x, y)` — mută bila cu `x` pixeli pe orizontală și `y` pixeli pe verticală (adică o alunecăm).

Proiect Minge



- `pos = canvas.coords(ball)` — aflăm poziția bilei: returnează o listă [`left`, `top`, `right`, `bottom`] adică [`stânga`, `sus`, `dreapta`, `jos`]. Exemplu inițial: [`5`, `5`, `60`, `60`].
 - `pos[0]` = stânga
 - `pos[1]` = sus
 - `pos[2]` = dreapta
 - `pos[3]` = jos
- Verificarea coliziunilor (bouncing):
 - ❖ `if pos[3] >= 400 or pos[1] <= 0: y = -y`
 - Dacă partea de jos a bilei (`pos[3]`) atinge sau trece de `400` (înălțimea canvas-ului) **sau** partea de sus (`pos[1]`) este `<= 0`, atunci învârtim direcția verticală: `y = -y`. Asta face bila să „sară” înapoi.
 - ❖ `if pos[2] >= 300 or pos[0] <= 0: x = -x`
 - Similar pe orizontală: dacă dreapta atinge `300` (lățimea), sau stânga e `<= 0`, schimbăm direcția pe orizontală.
 - ❖ Ideea e simplă: când lovește o margine, schimbăm semnul vitezei — de la `+` la `-` sau invers — ca și cum ar sări.

Bucă care mișcă bila



Proiect Minge



- `tk.update()` — spune ferestrei „procesează evenimentele și redesenează tot”. E ca și cum ai zice „ok, arată ce s-a schimbat”. Asta este necesar aici pentru ca schimbările să apară
- `time.sleep(0.025)` — mică pauză (25 milisecunde) pentru a controla viteza animației. Fără ea, bila s-ar mișca prea repede.
- `pass` — nu face nimic; e doar o linie care nu schimbă nimic (poate fi pusă ca loc-ținător).

`tk.mainloop()`



- `mainloop()` este metoda obișnuită care pornește „motorul” ferestrei și se ocupă de evenimente (click-uri, redesenări etc.). De obicei, la aplicațiile Tkinter pui toate setările și apoi `mainloop()` și el rulează singur.
- În acest cod, **nu se ajunge la `tk.mainloop()`** pentru că înainte avem `while True:` — o buclă infinită — care ține programul blocat acolo. Dar animația tot funcționează pentru că apelăm manual `tk.update()` în buclă pentru a actualiza fereastra.

Proiect Minge (AVANSAT)



```
import tkinter as tkr
import time
import random

"""Create Canvas"""
tk = tkr.Tk()
canvas = tkr.Canvas(tk, width=300,height=400)
canvas.grid()

"""Draw Ball"""
culori = ["light blue", "red", "yellow", "green", "orange",
"purple", "pink"]
ball=canvas.create_oval(5,5,60,60,fill="light blue")

x=1
y=3
```

Proiect Minge (AVANSAT)



```
"""Move Ball"""  
while True:  
    canvas.move(ball,x,y)  
    """Pos[Left,Top,Rigth,Bottom] """  
    pos = canvas.coords(ball)  
    if pos[3] >= 400 or pos[1]<=0:  
        y=-y  
        # schimbă culoarea la atingerea marginii sus/jos  
        canvas.itemconfig(ball, fill=random.choice(culori))  
    if pos[2]>=300 or pos[0]<=0:  
        x=-x  
        # schimbă culoarea la atingerea marginii stânga/dreapta  
        canvas.itemconfig(ball, fill=random.choice(culori))  
    tk.update()  
    time.sleep(0.025)  
    pass  
  
tk.mainloop()
```

Proiect Minge (AVANSAT)

```
import random
```

- `random` este o bibliotecă care ne lasă să alegem lucruri „la întâmplare”.
- Noi îl folosim ca să alegem **o culoare random** pentru bilă.

```
culori = ["light blue", "red", "yellow", "green",  
"orange", "purple", "pink"]
```

- Aici am scris **o listă** (o cutiuță în care punem mai multe lucruri).
- În cutiuță am pus nume de culori.
- Bila va alege una dintre aceste culori atunci când sare de margine.

```
canvas.itemconfig(ball, fill=random.choice(culori))
```

- `canvas.itemconfig(ball, fill=...)` înseamnă: „schimbă proprietatea bilei” (în acest caz, culoarea).
- `random.choice(culori)` alege o culoare la întâmplare din lista noastră.
- La fiecare atingere de **sus** sau **jos**, bila își schimbă culoarea.



Aplicație extra



Aplicația 1

Rezolvă câteva nivele din jocul de mai jos indicate de trainer:

<https://codecombat.com/>

